

Codeforces Mashup Generator

A Deep Dive, from zero prior knowledge

Explainer for Salman Adnan's portfolio

The whole project in four sentences

A study group wanted to run practice contests, but people kept being handed problems they had already solved, which defeats the point of a contest. This tool asks the public Codeforces website for its entire problem catalogue, then asks it for the full submission history of every group member, and throws away every problem that any member has already solved. It can also insist that at least one strong, trusted competitor has solved a problem before it is considered worth setting. What survives is written to a spreadsheet, grouped by difficulty.

Contents

1	Orientation: what this thing actually is	2
1.1	The vocabulary you need first	2
1.2	The problem being solved	3
2	The shape of the repository	3
2.1	The one file that should not be there	4
3	Architecture	5
3.1	The pipeline	5
3.2	The idea that makes it work	6
4	Walkthrough of every function	6
4.1	parse_args: the entire user interface	6
4.2	fetch_user_status: pagination, and knowing when to stop	7
4.3	fetch_problemset: one request, the whole catalogue	9
4.4	collect_solved: two callers, one rule	9
4.5	get_filtered_problems: bucketing, then the three gates	10
4.6	passes_tag_filter: four modes, one asymmetry	11
4.7	pick_choices_per_rating: two lines, one real weakness	13
4.8	save_to_excel: flatten and write	13
5	Dependencies	14
6	Verified behaviour	14
7	Performance, measured	15
8	Honest assessment	15
8.1	The one that actually matters	15
8.2	The rest, in descending order of consequence	16
9	What to say about this project	17

1 Orientation: what this thing actually is

1.1 The vocabulary you need first

This project sits inside a hobby, competitive programming, that has its own dense vocabulary. Nothing below is assumed.

Jargon: Competitive programming

A sport where people solve small, self-contained algorithm puzzles against the clock. You are given a problem statement, you write a program, you submit it, and an automated judge either accepts it or rejects it.

Jargon: Codeforces

One of the largest websites hosting this sport. It stores tens of thousands of problems and a permanent record of every submission every user has ever made. It exposes that record publicly through an API (defined below), which is the only reason this project can exist.

Jargon: API (Application Programming Interface)

A door into a website meant for programs rather than people. Instead of a page of styled text for a human to read, you request a URL and get back structured data for a program to read. Codeforces' API is public: no key, no password, no account needed.

Jargon: JSON

The format that structured data comes back in. It is text made of nested lists and labelled fields, which a program can turn directly into its own native lists and dictionaries. A Codeforces problem arrives as JSON that looks, in spirit, like: contest number 2236, letter A, name "Games on the Train", rating 800, tags greedy and math.

Jargon: Rating (of a problem)

Codeforces labels each problem with a difficulty number, roughly 800 (easiest) to 3500 (hardest). This project's default range is 800 to 2000 in steps of 100. Rating is the axis the whole tool is organised around: the output is "give me N problems at each of these difficulties".

Jargon: Tags

Codeforces labels each problem with the techniques needed to solve it: **greedy**, **dp** (dynamic programming), **bitmasks**, **graphs**, and so on. A problem usually carries several at once.

Jargon: Mashup

A Codeforces feature that lets anyone assemble a private practice contest out of existing problems, rather than writing new ones. The tool's whole job is choosing which existing problems go in. It does not create the mashup itself; a human takes the spreadsheet and sets it up.

Jargon: Verdict, and OK

The judge's ruling on a submission. **OK** means accepted, that is, actually solved. Every other verdict (wrong answer, time limit exceeded, compile error) means the person tried and failed.

This distinction is load-bearing: the tool counts a problem as “solved by you” only on an OK verdict, so problems you attempted and failed are still fair game.

1.2 The problem being solved

The tool answers one question, per difficulty band:

Which Codeforces problems has nobody in my group solved, that a strong competitor has solved?

The two halves of that question do different jobs, and it is worth being precise about which is which:

- The **exclude** half is the point of the tool. It enforces fairness. If any group member has an accepted submission on a problem, that problem is out.
- The **include** half is a quality heuristic, and only a heuristic. The reasoning encoded in the default list is that if a well-known strong competitor has bothered to solve a problem, it is probably a real problem worth setting rather than an obscure or broken one. This is a proxy, not a measure of quality, and section 8 is blunt about what it costs.

The include filter is expensive for what it buys

The include filter’s stated purpose is a rough quality signal. Codeforces already ships a direct quality signal in the problemset response itself: `solvedCount`, the number of people who have solved each problem, which arrives free with data the tool already downloads. The tool instead downloads the complete lifetime submission history of 46 separate people to approximate the same idea. The README’s own “What I’d do differently” section names this, which is to its credit. I measured the actual cost in section 7: it is worse than the README implies.

2 The shape of the repository

The entire project is one Python file. Everything else is packaging, documentation, or, in one case, dead weight.

```

codeforces-mashup-generator
├── generate_mashup.py    the entire program, 228 lines
├── requirements.txt     three dependencies
├── all_users.txt        dead: never read by anything
├── .env.example         states that no env vars exist
├── .gitignore
├── LICENSE              PolyForm Strict 1.0.0
├── README.md
├── docs
│   ├── screenshots
│   │   └── app.png
│   └── explainers
│       ├── deep-dive.tex    this document
│       └── brief.tex

```

Figure 1: The tracked files. One script, no package structure, no test suite.

Why a single file is the right call here

A beginner is often told to split code into modules. That advice has a threshold. This program is a linear pipeline with five functions, no state that outlives a run, and no second consumer of any of its parts. Splitting it across files would add navigation cost and buy nothing. The absence of structure here is proportionate, not lazy.

2.1 The one file that should not be there

`all_users.txt` is dead data, and I can prove it is stale

`all_users.txt` sits at the project root holding two Python lists, `solver_handles` and `exclude_handles`. It looks like configuration. It is not. `generate_mashup.py` performs **no file I/O whatsoever**: it has no `open()` call, no file read, and no import of this file. The script's own `DEFAULT_INCLUDE_USERS` and `DEFAULT_EXCLUDE_USERS` constants are what actually run. Deleting `all_users.txt` would change nothing.

I compared the two mechanically rather than by eye, and the result pins down its history:

- The include lists are **byte-for-byte identical**: 46 handles, same order.
- The exclude lists are **not**. The file has 28 handles; the script has 34. The file's 28 are a **strict subset** of the script's 34, missing exactly these six: `salmanadnan2025`, `Aritro_`, `Drakozs`, `chpp`, `Vyaduct`, `chiru200513`. Nothing is in the file that is not in the script.

A strict subset in one list and an exact match in the other is the signature of a snapshot that was forked and then left behind while the live copy kept growing by six members. **Confirmed:** the file is unread and its exclude list is a strict subset. **Inferred:** that it is a pre-refactor leftover rather than something intended for future use. The code cannot tell us intent. It should be deleted; a file that looks like config but is not is a trap for the next reader, and this is the second document to have to explain it.

3 Architecture

3.1 The pipeline

The program is a straight line. Data enters from two API endpoints, gets reduced to sets, gets filtered, gets truncated, and lands in a spreadsheet. There are no branches in the overall shape, no loops back, and no persistence between runs.

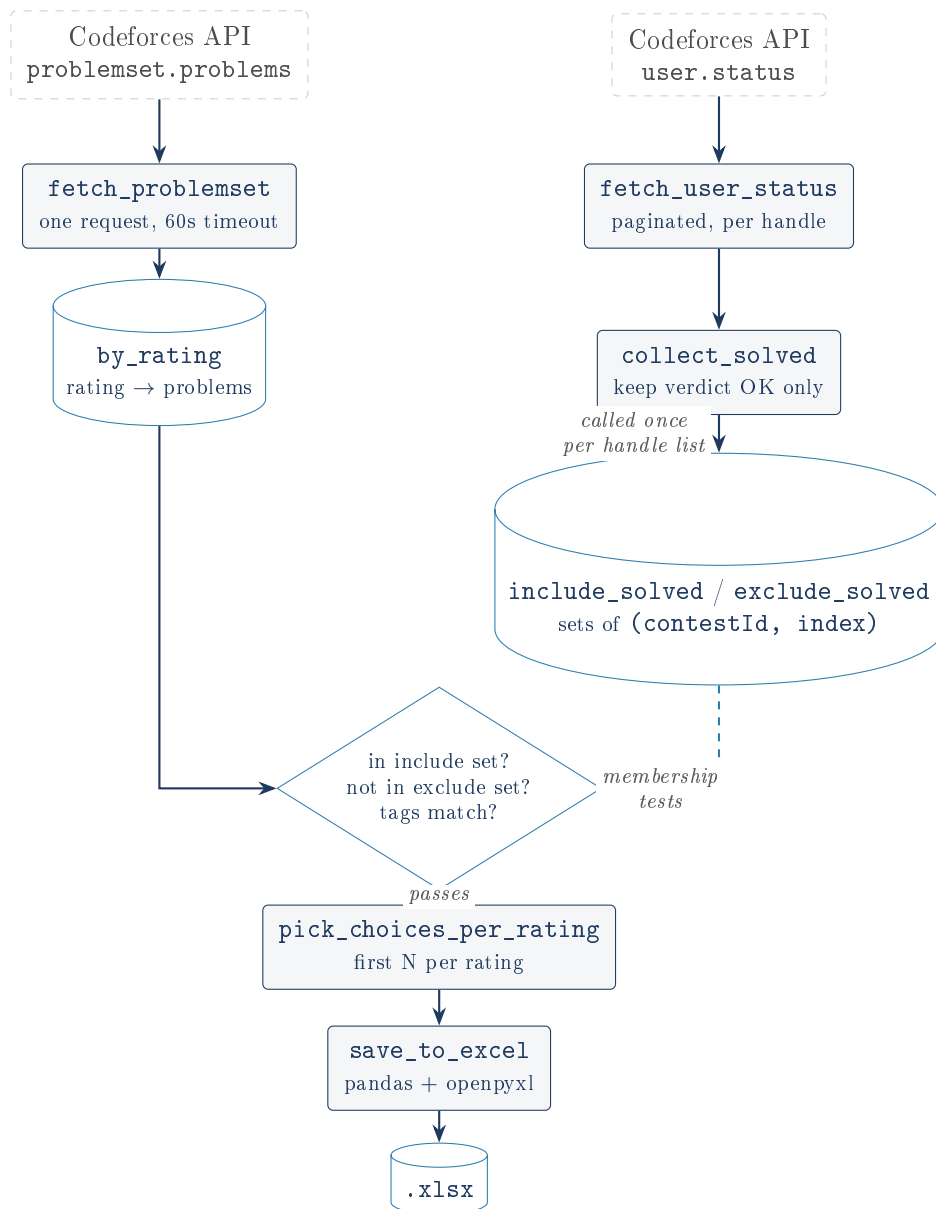


Figure 2: The end-to-end pipeline, read top to bottom. The left lane fetches the catalogue, the right lane fetches submission history and reduces it to sets of keys; they meet at the filter. Dashed arrows are set-membership tests, solid arrows are data movement.

3.2 The idea that makes it work

One identity, normalised from two different envelopes

This is the single design decision worth defending in an interview, and it is genuinely the crux of the project.

Codeforces hands you the same logical thing, “a problem”, in two **different shapes** depending on which endpoint you ask.

- `problemset.problems` returns problem objects directly: `contestId` and `index` sit at the top level.
- `user.status` returns *submission* objects, which wrap the problem one level down under a `problem` key, alongside the verdict and timestamp.

You cannot compare those two objects. They are not the same shape, and even the parts that overlap carry extra fields that differ per submission. So the code picks a **canonical identity** that both can be reduced to: the tuple `(contestId, index)`, for example `(2236, "A")`. Every submission collapses to that tuple, every catalogue entry collapses to that tuple, and now the two sources are directly comparable.

Once both sides speak the same identity, the entire filtering question becomes set arithmetic: is this key in the include set, and is it absent from the exclude set. That is why the code has no nested search anywhere.

Jargon: Set, and why it is used here

A set is a collection that answers “is this item in you?” in effectively constant time, no matter how large it is, because it is indexed by a hash rather than scanned. A plain list answers the same question by checking every element in turn. With roughly 11,000 problems tested against hundreds of thousands of submissions, the difference is between a run that finishes and one that does not. The code builds `include_solved` and `exclude_solved` as sets once, then does one $O(1)$ lookup per problem.

4 Walkthrough of every function

The file has eight top-level definitions. I take them in dependency order rather than file order.

4.1 `parse_args`: the entire user interface

The tool has no configuration file and no interactive prompts. Every knob is a command line flag, defined in one `argparse` parser.

Jargon: `argparse`

Python’s standard library for command line interfaces. You declare the flags you accept and their defaults; it handles parsing, type conversion, validation, and generating the `--help` text for free. The `ArgumentDefaultsHelpFormatter` used here also prints each default into the help output, so `--help` is self-documenting.

Flag	Default	Effect
<code>--include-users</code>	46 handles	Problem must be solved by at least one of these
<code>--exclude-users</code>	34 handles	Problem must be solved by none of these
<code>--no-include-filter</code>	off	Skip the include stage entirely, and skip its download
<code>--no-exclude-filter</code>	off	Skip the exclude stage entirely
<code>--ratings</code>	800..2000 by 100	Which difficulty bands to fill
<code>--choices-per-rating</code>	10	How many options per band
<code>--include-tags</code>	(empty)	Tags to require, per <code>--tag-mode</code>
<code>--exclude-tags</code>	(empty)	Tags that disqualify a problem outright
<code>--tag-mode</code>	all	off, some, all, or strict
<code>--lang</code>	en	Statement language passed to the API
<code>--delay</code>	0.5	Seconds slept between API calls
<code>-o, --output</code>	filtered_problems.xlsx	Output filename

Table 1: The complete CLI surface. Verified against the source, not the README.

A default that contradicts itself, harmlessly

`--tag-mode` defaults to `all`, but `--include-tags` defaults to empty, and `passes_tag_filter` only consults the mode when the include-tags list is non-empty. So the default mode is inert: it describes how to match a list that is not there. This is not a bug, and it is arguably the friendliest default (the moment you pass tags, “all” is the intuitive reading), but it does mean the `--help` output advertises an active default that does nothing on a bare run.

4.2 `fetch_user_status`: pagination, and knowing when to stop

This is the most intricate function in the file, because it is the only one with a loop whose exit condition is not obvious.

Jargon: Pagination

An API that would otherwise return an enormous response instead returns it in slices. You ask for a window (“give me 10,000 records starting at record 1”), then ask again for the next window, until you have everything. Codeforces exposes this through `from` and `count` parameters on `user.status`. The awkward part is always the same: knowing when you have reached the end.

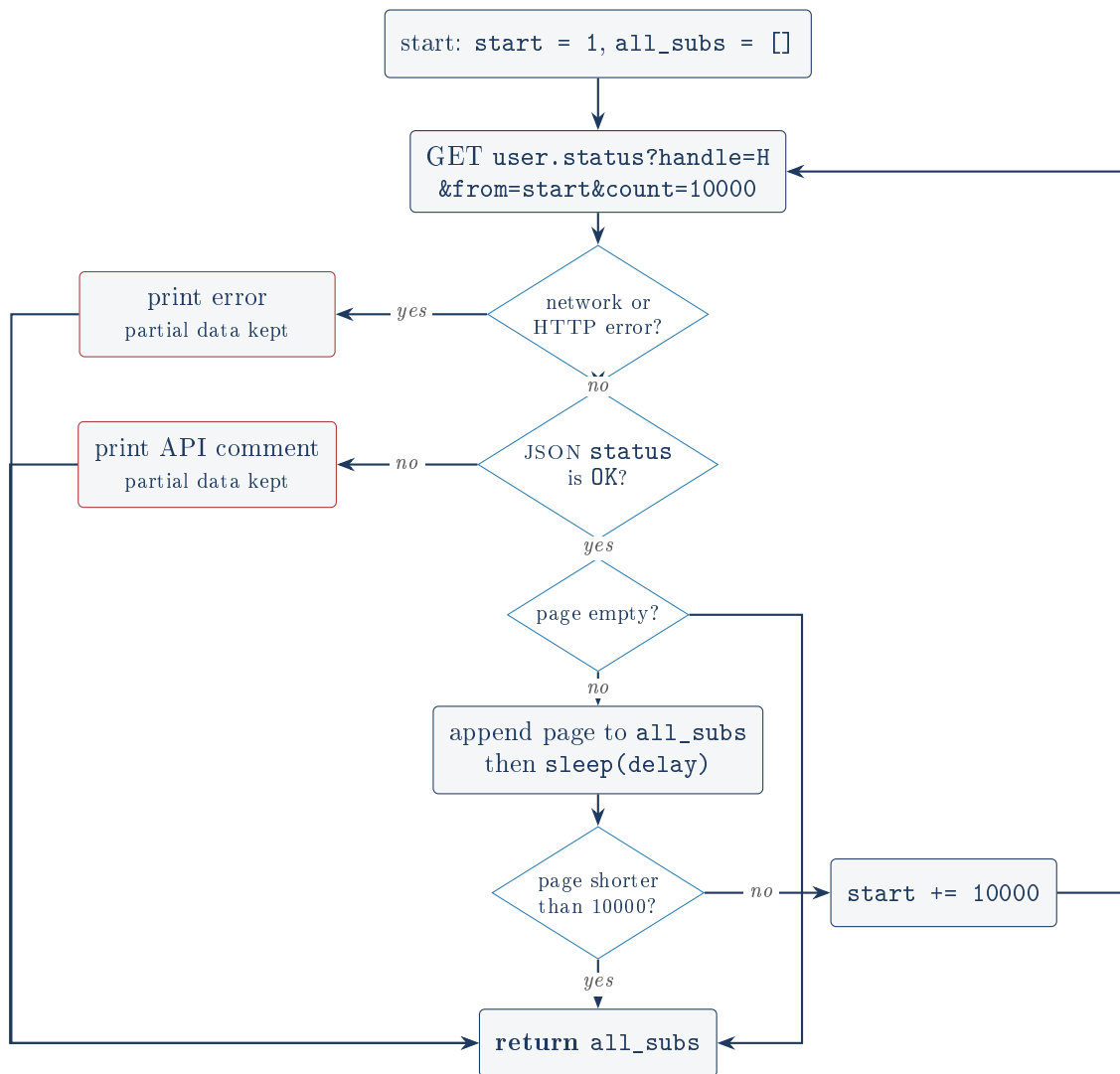


Figure 3: `fetch_user_status` as a state machine. Note that all three red and grey exits converge on the same return, and none of them signals failure to the caller. That convergence is the root of the finding in section 8.1.

```

1      subs = data.get('result', [])
2      if not subs:
3          break
4      all_subs.extend(subs)
5      time.sleep(delay)
6      if len(subs) < batch_size:
7          break
8      start += batch_size

```

Listing 1: The stop condition, the part worth reading twice

There are two stop conditions, and they are not redundant:

1. **An empty page** means we asked past the end. This catches the exact-multiple case, where a user has precisely 20,000 submissions and the second page is full.
2. **A short page** means this page *is* the end. This is the common case, and it saves one wasted round trip that would have returned nothing.

Why “short page means done” is safe here

This inference is not universally valid. It is valid against an API that only returns a short page when it has run out of records, and never returns a short page for its own internal reasons (throttling, partial results). Codeforces behaves that way, so the optimisation holds. I confirmed the branch is real and load-bearing by measuring live: `tourist` returned 5459 records on page one, which is shorter than the 10,000 requested, so the loop stopped after a single request instead of making a second one.

4.3 `fetch_problemset`: one request, the whole catalogue

The counterpart is trivial by comparison: a single request for the entire problem catalogue, no pagination, a 60 second timeout rather than 30 because the response is much larger. On success it sleeps and returns the problems list; on any failure it prints and returns an empty list.

Failing to fetch the catalogue produces a successful-looking run

If this request fails, the function returns []. That empty list flows through `get_filtered_problems`, which happily filters nothing, into `save_to_excel`, which prints “No problems to save.” and returns. Then `__main__` prints “Done.” and the process exits with status 0, the code meaning success. A scheduled job wrapping this script would see a green tick and no spreadsheet. The word “Done.” printing after a total failure to fetch anything is the clearest example in the codebase of the exit-code problem discussed in section 8.

4.4 `collect_solved`: two callers, one rule

```

1 def collect_solved(handles, delay):
2     """Set of (contestId, index) pairs with an OK verdict across the given handles."""
3     solved = set()
4     for u in handles:
5         for s in fetch_user_status(u, delay):
6             if s.get('verdict') == 'OK':
7                 prob = s.get('problem', {})
8                 cid = prob.get('contestId') or s.get('contestId')
9                 idx = prob.get('index')
10                if cid and idx:
11                    solved.add((cid, idx))
12    return solved

```

Listing 2: The normalisation step, in full

Nine lines, and three separate decisions inside them:

1. `verdict == 'OK'` is the fairness rule itself. A member who attempted a problem and failed has not solved it, and the problem stays eligible. Every other verdict falls through.
2. `prob.get('contestId') or s.get('contestId')` is defensive normalisation. The contest id normally lives on the nested problem, but not always, so the code falls back to the submission envelope. This is a real quirk of the API surface, not speculative hardening.
3. `if cid and idx` refuses to build a half-formed key. Without it, an entry missing an index would insert (2236, None) into the set, a key that can never match a real problem but silently pollutes the data.

Why this function exists at all

Include and exclude need the identical computation over different handle lists. The README records that this was originally one loop copy-pasted twice, so a fix to the verdict rule had to be made in two places or it was wrong in one. Parameterising by `handles` and calling it twice means the OK rule, the fallback, and the guard each exist exactly once. This is the clearest refactor in the project, and it is worth naming as such: the duplication was not merely ugly, it was a correctness hazard.

A falsy guard where a None check was meant

`if cid and idx` rejects any value Python considers falsy, not just missing ones. A contest id of 0 would be silently discarded. No Codeforces contest has id 0, so this is harmless in practice and I am flagging it as a code-reading observation rather than a defect: the guard expresses “is present” using a test that means “is truthy”, and those two only coincide because of a fact about the data that is nowhere written down.

4.5 `get_filtered_problems`: bucketing, then the three gates

This orchestrates everything. First it buckets the catalogue by rating:

```
1 by_rating = {r: [] for r in args.ratings}
2 for p in problems:
3     r = p.get('rating')
4     if r in by_rating:
5         by_rating[r].append(p)
```

Pre-seeding the dictionary is doing two jobs

`by_rating` is created with a key for every requested rating and an empty list behind it, *before* any problem is examined. That means the subsequent `if r in by_rating` test is simultaneously “do we want this rating?” and “where does it go?”. One dictionary lookup replaces both a membership test against the requested ratings and a create-if-absent. It also guarantees a requested rating with zero matching problems still appears downstream as an empty bucket rather than vanishing.

Problems whose rating is unset are dropped here automatically, because `p.get('rating')` returns `None` and `None` is not a key in `by_rating`. That case is real: I measured the live catalogue at 11,295 problems, of which **289 carry no rating at all**. The `.get` is doing genuine work.

Then the two solved-sets are built, each guarded so that disabling a filter also skips its download entirely, and each problem faces three gates:

```
1 key = (p['contestId'], p['index'])
2 if (
3     (args.no_include_filter or key in include_solved)
4     and (args.no_exclude_filter or key not in exclude_solved)
5     and passes_tag_filter(p)
6 ):
7     valid.append(p)
```

Listing 3: The core predicate

The `or` short-circuits are how a disabled filter is expressed: if `no_include_filter` is true, the whole clause is true without ever consulting the (empty) set. Correct, and it reads cleanly.

Inconsistent defensiveness on the same two fields

`collect_solved` carefully uses `.get` with a fallback for `contestId` and guards against missing values. Twenty lines later, `get_filtered_problems` uses direct subscripting, `p['contestId']` and `p['index']`, which raises `KeyError` and kills the run if either is absent. The same two fields are treated as untrustworthy in one function and guaranteed in the other.

I checked whether this can actually fire: across all 11,295 problems in the live catalogue, **zero** are missing `contestId` and **zero** are missing `index`. So the crash is unreachable today. The defect is the inconsistency itself, not a live bug: a reader cannot tell from the code which function is right about the API's guarantees, and the answer turns out to be that the defensive one is defending against a quirk of *submissions*, not of the catalogue. That is a reasonable distinction which the code never states.

4.6 passes_tag_filter: four modes, one asymmetry

Defined as a closure inside `get_filtered_problems` so it can read `args` without being passed it.

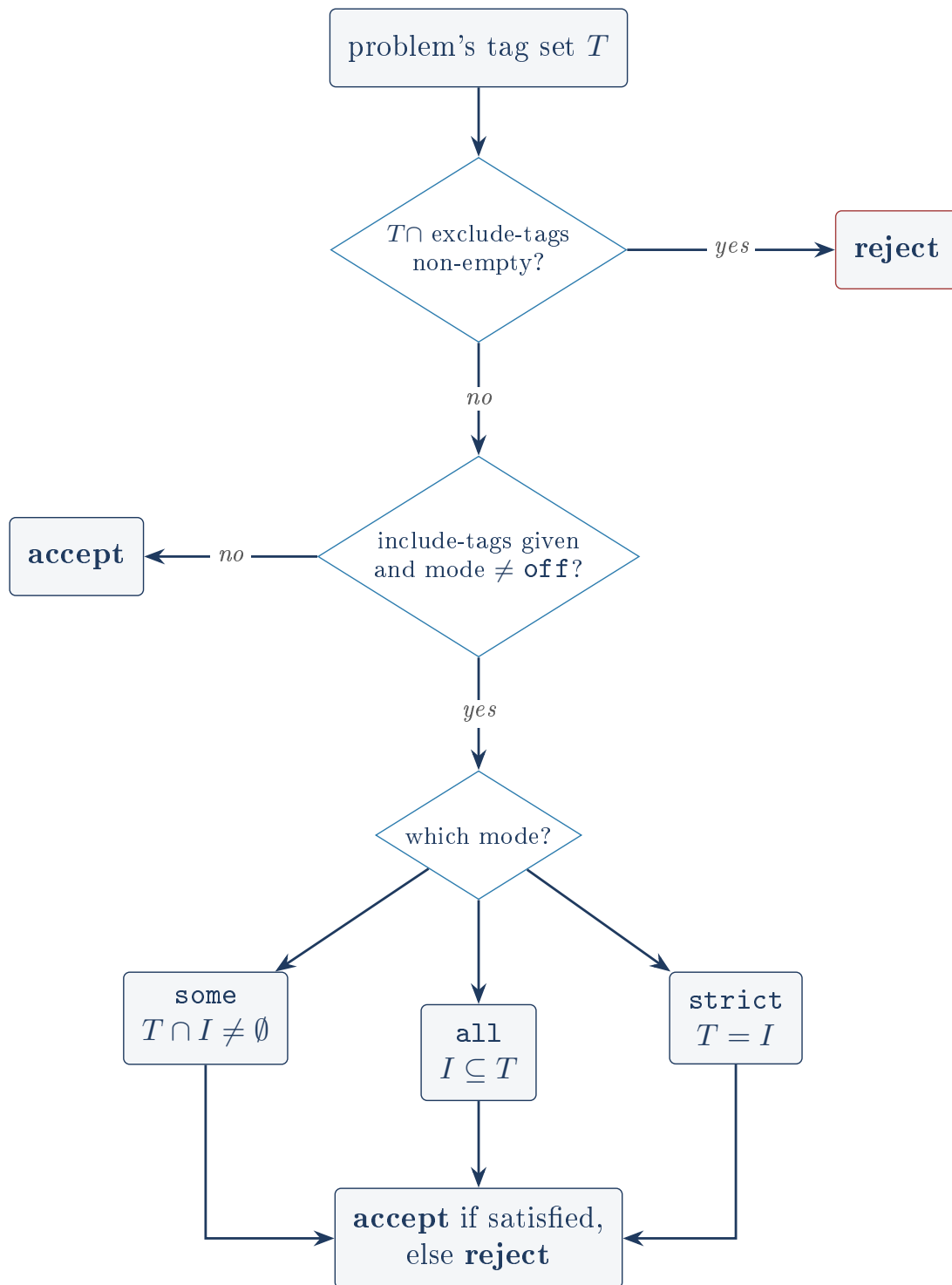


Figure 4: Tag filtering. I is the include-tags set. Note the asymmetry at the top: the exclude-tags gate runs unconditionally, before and independent of the mode.

The asymmetry is deliberate and correct

`--tag-mode off` disables the *include* tags but leaves `--exclude-tags` fully active. That is the right semantics. “Mode” answers the question “how should I match the tags you asked for?”, which is meaningless for tags you asked to avoid: there is only one way to avoid a tag. Reading the code, the exclude check sits above the mode logic entirely, so the two can never interfere. Someone could reasonably expect `off` to mean “no tag filtering at all”, and it does not; the README documents the behaviour correctly.

The three include modes, precisely:

- **some**: at least one requested tag present. The loosest. *Give me anything greedy-ish.*
- **all**: every requested tag present, extras allowed. *Must be greedy AND maths, whatever else it is.*
- **strict**: the problem's tag set equals the requested set exactly, no extras. *Purely greedy, nothing else.* On real data this is extremely restrictive, since most problems carry three or more tags.

4.7 `pick_choices_per_rating`: two lines, one real weakness

```
1 def pick_choices_per_rating(filtered, n):
2     selection = {}
3     for r, plist in filtered.items():
4         selection[r] = plist[:n]
5     return selection
```

“Selection” is a truncation, and it is biased, not neutral

`plist[:n]` takes the first N in whatever order the API returned. It does not sample. Codeforces returns the problemset newest-first, so this systematically hands you the most recent problems and never surfaces an older classic, and two runs with the same filters produce nearly the same sheet, which undercuts the point of a tool for a recurring study group.

This is not theory. In my live run (section 6) I asked for three problems each at ratings 800, 1200 and 1600. The contest ids that came back were 2237, 2236, 2236, 2227, 2189, 2178, 2207, 2172, 2157: all from a narrow, recent band, out of a catalogue spanning contest ids from the low hundreds upward. The bias is total, not marginal.

The README's “What I'd do differently” names this and proposes `random.sample` with a seed. That is the correct fix and it is about four lines. Worth being ready to say in an interview why it was not done: the honest answer is that the tool met the group's need as written, not that the problem was unrecognised.

4.8 `save_to_excel`: flatten and write

Takes the nested dictionary of rating to problem-list and flattens it to one row per problem, deriving a clickable link from the identity tuple. It bails out early with “No problems to save.” rather than writing an empty file.

Jargon: `pandas` and `DataFrame`

`pandas` is Python's standard library for tabular data. A `DataFrame` is a table in memory with named columns, essentially a spreadsheet as a variable. Here it is used at its most basic: build a list of dictionaries where each dictionary is a row, hand it to `pd.DataFrame`, and call `.to_excel`. The heavy lifting of writing the actual `.xlsx` binary format is delegated to `openpyxl`.

`pandas` is a heavy dependency for what it does here

The only `pandas` features used are “list of dicts becomes a table” and “table becomes `.xlsx`”. `openpyxl` alone can do that, and `pandas` depends on `numpy`, so this pulls tens of megabytes of scientific computing to write a nine-row sheet. It is a defensible convenience choice, and `to_excel` genuinely is one line, but it is worth being able to name the tradeoff rather than being caught by it.

5 Dependencies

Package	Pin	Role
requests	>=2.31,<3	Every HTTP call to the Codeforces API. Used for its timeout support and <code>raise_for_status</code> .
pandas	>=2.0,<3	Builds the table and writes the sheet.
openpyxl	>=3.1,<4	The actual <code>.xlsx</code> writer. Never imported directly; pandas requires it to be installed to write Excel.

Table 2: All three dependencies. Verified installable and working on Python 3.12.3.

Why openpyxl appears in requirements but never in the code

A beginner grepping for `import openpyxl` will not find it and may conclude the dependency is spurious. It is not. `pandas.to_excel` is a thin front end that looks up an engine at runtime and delegates; without `openpyxl` installed it raises at the moment of writing, after all the API downloading is already done. Pinning it explicitly is correct: it is a real runtime requirement that the import graph does not reveal.

The pins are compatible-range style: at least a known-good minor, below the next major. This is the sensible middle ground between unpinned (breaks silently when an upstream major lands) and exact pins (rots, needs constant bumping).

6 Verified behaviour

I did not take the README's claims on trust. I built a clean virtual environment on Python 3.12.3, installed the three pinned dependencies, and ran the exact command the README documents, against the live Codeforces API.

```

1 python generate_mashup.py \
2   --include-users ksun48 \
3   --exclude-users salmanadnan2025 Musab1Blaser \
4   --ratings 800 1200 1600 \
5   --choices-per-rating 3 \
6   --include-tags greedy --tag-mode some \
7   -o mashup_test.xlsx

```

Listing 4: The command from the README, run verbatim

README claim	Measured	Verdict
Takes about 14 seconds	12.1 seconds	holds
Produced 9 rows	9 rows	holds
3 per rating	3 at each of 800, 1200, 1600	holds
All tagged greedy	9 of 9 carry greedy	holds
46 trusted handles, 34 group members	46 and 34, no duplicates	holds
Bad handles print and skip, sheet still generates	reproduced with a fabricated handle	holds

Table 3: Every checkable README claim, checked. This README is unusually accurate; nothing in it was found to be overstated.

The output columns are `rating`, `contestId`, `index`, `name`, `tags`, `link`, exactly as documented. A representative row: rating 800, contest 2236, index A, “Games on the Train”, tags `greedy`, `math`, with a working problemset link.

I also confirmed the include and exclude default lists are disjoint (no handle appears in both), so the pathological case of a handle being downloaded twice and simultaneously qualifying and disqualifying every problem it has solved does not arise on a default run. Nothing in the code prevents it if a user constructs it by hand.

7 Performance, measured

The README says a default run pulls “hundreds of megabytes”. I checked rather than repeating it. Fetching the first page of submissions for three of the default trusted handles:

Handle	Records, page 1	Wire size
tourist	5,459	3.1 MB
ksun48	9,101	5.2 MB
jiangly	10,000 (page 1 of more)	5.6 MB

Table 4: Measured live. `jiangly` hit the 10,000 cap, so that 5.6 MB is a floor, not a total.

Averaging those three and extrapolating across the 46 default include handles gives roughly **214 MB, as a lower bound**, since it counts only one page each and ignores the 34 exclude handles entirely. The README’s “hundreds of megabytes” is **substantiated, and if anything conservative**. All of it is downloaded to answer a set-membership question, and none of it is cached: run the tool twice in a row with identical arguments and it downloads everything again.

Jargon: Caching

Storing the result of expensive work so a later run can reuse it instead of redoing it. Here, writing each handle’s solved-set to a small file on disk would turn the second run of a day from a 200 MB download into reading a few kilobytes. The tool does none of this, which is fine for a script run occasionally by one person and would be the first thing to fix if it ran more often.

8 Honest assessment

The code is clean, readable, and does what it claims. The findings below are real, and several are already named in the README’s own “What I’d do differently”, which is a point in the author’s favour: he found most of them himself.

8.1 The one that actually matters

A failed fetch silently voids the fairness guarantee

This is the most serious defect in the project, because it defeats the tool’s entire purpose while looking like it worked.

Follow figure 3: every error path in `fetch_user_status` prints a message and returns whatever it has, which for a failure on the first request is an empty list. `collect_solved` cannot distinguish “this member has solved nothing” from “we failed to find out what this member has solved”. Both are the empty set.

So if a network blip, a rate limit, or a renamed handle causes one group member’s fetch to fail, that member’s solved problems are treated as unsolved, they sail through the exclude

filter, and the member is handed a problem they have already done. That is precisely the scenario the tool exists to prevent. The run prints one line among many and exits 0.

I reproduced the mechanism live: an invented handle in `--exclude-users` produced `Error fetching submissions for ...: 400 Client Error`, and then the sheet generated normally and the process exited 0. The README notes that this happened for real, with two stale handles in the default exclude list returning 400.

The severity is asymmetric and the code does not reflect that. A failed *include* fetch only costs you some candidate problems, which is a quality loss. A failed *exclude* fetch costs you the guarantee, which is a correctness loss. They are handled identically. A failed exclude fetch should abort the run. The README's own "What I'd do differently" reaches the same conclusion, in the same words: fairness is the whole point.

8.2 The rest, in descending order of consequence

No tests, at all

There is no test file in the repository and no test dependency. Every claim about this code, including every one in this document, had to be established by reading it or by running it against the live API. The pure functions are the easy, obvious targets: `passes_tag_filter`'s four modes and `pick_choices_per_rating` are completely deterministic, take plain data, touch no network, and could be pinned down in a few dozen lines. `collect_solved`'s envelope-normalisation is exactly the kind of fiddly logic that regresses silently. As it stands, the only way to know a refactor was safe is to run the whole thing and eyeball a spreadsheet.

Everything exits 0, and reports through print

There is no logging and no exit code discipline. Success, partial data, a completely failed catalogue fetch, and a run that saved nothing all exit 0 and all report through `print` to stdout. Errors are interleaved with results on the same stream. The script is therefore not safely automatable: nothing downstream can detect that it failed. Adding logging to stderr and a nonzero exit when no rows were written is small work with a large payoff, and the README names it.

Dead data at the project root

`all_users.txt`, covered in full earlier: unread by any code, and demonstrably a stale snapshot of the script's own constants, six handles behind on the exclude list. It should be deleted.

The default run is close to unusable, and the defaults advertise it

A bare `python generate_mashup.py` downloads the full submission history of 80 handles, on the order of a quarter of a gigabyte at minimum (section 7), with a 0.5 second sleep between every page, to produce a spreadsheet. The README warns that defaults "take a while". That warning is doing a lot of work: this is measured in many minutes, and the tool's own documentation only ever demonstrates it on a cut-down example. The defaults reproduce a specific study group's configuration, which is defensible as provenance, but it means the out-of-box experience is the slowest possible path.

A personal handle is baked into a public default

`salmanadnan2025` sits in `DEFAULT_EXCLUDE_USERS` in a public repository. That is intentional and harmless (he was in the study group, and it is a public handle on a public site), but it is

worth being conscious that the shipped defaults are one particular group's roster, including the author's own handle, rather than anything general. Anyone cloning this runs a filter built around 34 strangers unless they pass their own flags.

The `--delay` flag guards the wrong boundary

`--delay` sleeps between pages within a handle and after the problemset fetch, but Codeforces rate limits per client, not per handle. The sleep placement is a reasonable approximation and it evidently works, but it is not derived from the published limit. If a run ever does get throttled, the failure mode is the one in section 8.1: a non-OK response, a printed line, and a silently weakened exclude set.

9 What to say about this project

The defensible core

Strip away the polish and there is one genuinely good idea here, and it generalises well beyond Codeforces: **when an API hands you the same entity in two different envelope shapes, do not try to compare the objects. Choose a canonical identity, reduce both sides to it, and let set arithmetic do the rest.** That is what makes the include and exclude filters line up, it is why there is no nested search in the code, and it is the thing worth leading with.

The second defensible point is the `collect_solved` extraction: recognising that two copy-pasted loops were not merely repetitive but a correctness hazard, since the verdict rule could drift between them, and collapsing them into one parameterised function.

The honest framing of the whole project: it is a small, well-written tool that solved a real problem for a real group, with one significant unfixed flaw (a failed exclude fetch silently voids the guarantee), one significant inefficiency (downloading hundreds of megabytes to answer a membership question that `solvedCount` would answer for free), and no tests. The author identified most of that himself before anyone asked, which is the best thing in the repository.